

FPGA Kernel Optimization

Advanced Computer Architecture – Lab 3.4

Summer Semester 2026 — Übung 1 — Group 1

Daniel Wilger (4757813)

Daniel Bayer (4760237)

Arjan Siddhpura (3707267)

1 Hardware Platform

The system is built on the Zybo Z7 FPGA board featuring a Xilinx Zynq-7000 SoC. The MicroBlaze soft processor is instantiated in programmable logic with a configurable microarchitecture. Figure 1 shows the block diagram.

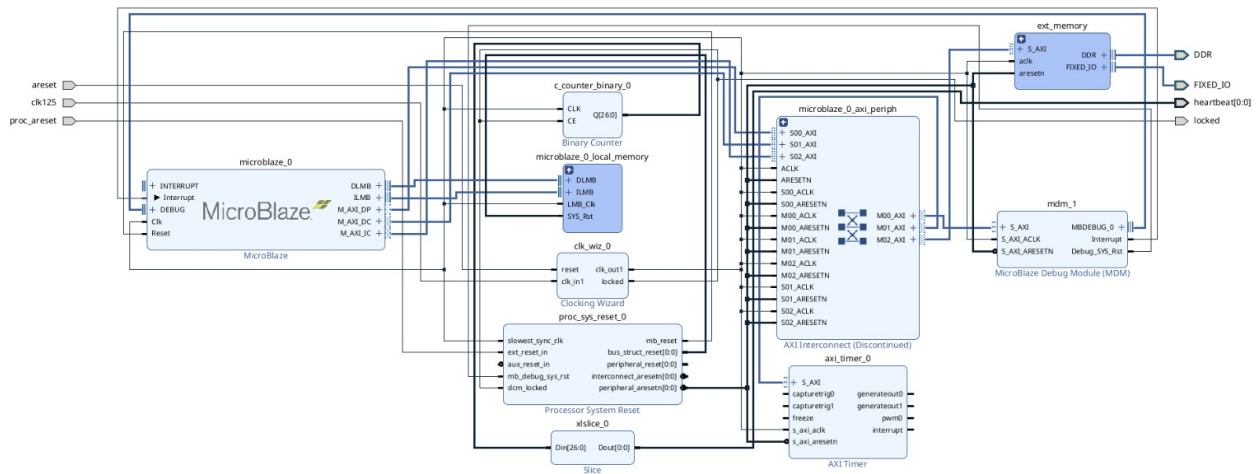


Figure 1: Vivado block diagram: MicroBlaze with local memory, AXI interconnect, DDR3 external memory controller (via ZYNQ PS), AXI Timer, binary counter (heartbeat), and MDM debug module.

Two hardware configurations were used to isolate the impact of microarchitectural features:

Table 1: MicroBlaze configuration comparison

Feature	Unoptimized	Optimized
Implementation Optimization	Extended (5-stage pipeline)	Extended (5-stage pipeline)
Instruction Cache (8 KB)	Disabled (SW)	Enabled
Data Cache (8 KB)	Disabled (SW)	Enabled
Floating Point Unit	None	Extended
Integer Multiplier	None	32-bit (MUL32)
Barrel Shifter	Disabled	Enabled
Integer Divider	Disabled	Enabled
Pattern Comparator	Disabled	Enabled
Branch Target Cache	Disabled	512 entries

Both configurations use the 5-stage pipeline (Extended implementation), as this was the default from the initial lab setup. The external DDR3 memory (512 MB, accessed via the ZYNQ PS HP0 AXI port) stores all program data and code. The heap was configured to 128 KB in the linker script to accommodate the test arrays.

2 Methodology

2.1 Experimental Design

To isolate the contribution of each optimization category, five configurations were tested per kernel, progressively enabling optimizations:

1. **Baseline:** -O0, original code, hardware unoptimized
2. **Compiler:** -O2, original code, hardware unoptimized
3. **Hardware:** -O0, original code, hardware optimized
4. **Software:** -O0, optimized code, hardware optimized
5. **All combined:** -O3, optimized code, hardware optimized

This progression reveals: compiler-only gains (1→2), hardware-only gains (1→3), software restructuring gains (3→4), and the synergistic combination (1→5). An additional test without the FPU was performed for Kernel 4 to quantify the FPU impact on floating-point workloads.

2.2 Measurement Protocol

Execution time was measured using the AXI Timer (64-bit counter at the MicroBlaze core clock frequency). Before each measurement, `Xil_DCacheInvalidate()` ensures a cold-cache start for fair comparison. Array size is 4096 elements for all kernels (128 for Kernel 2 to prevent overflow). All arrays are allocated on DDR3 via `malloc()` and initialized with pseudo-random data (`srand(42)`). Most measurements were performed as single runs; spot-checked repetitions showed variation below 0.0001 s, confirming high reproducibility of the AXI Timer on this deterministic (no-OS) platform.

2.3 Software Optimization Strategy

Figure 2 summarizes the optimization approach applied to each kernel.

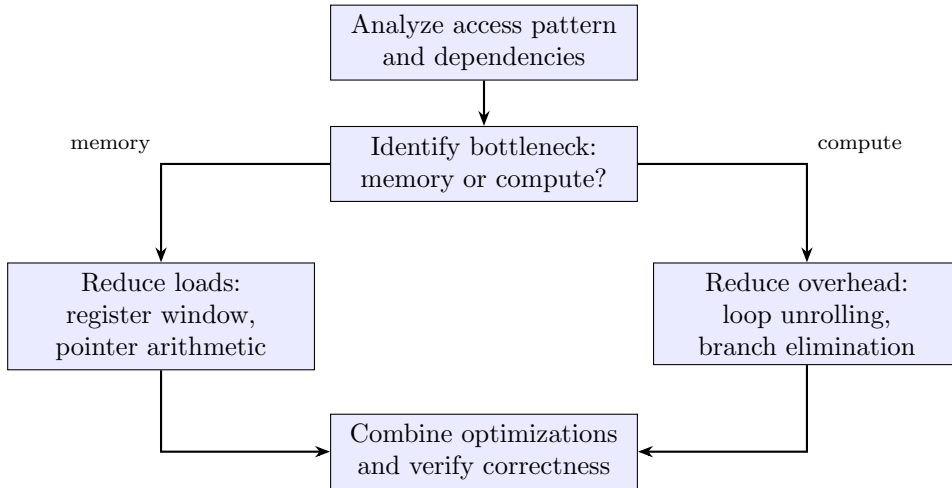


Figure 2: Kernel optimization decision flow.

Kernel 1 ($A[i] += A[i+offset]$): Stride-1 access pattern, memory-bound. Optimized with pointer arithmetic (eliminates index scaling) and $4\times$ loop unrolling (reduces branch overhead by 75%).

Kernel 2 ($A[i] = A[i-1] + A[i-2]*A[i-3]$): Serial loop-carried dependency limits parallelism. Optimized with a register sliding window that eliminates 2 of 3 loads per iteration by keeping $A[i-1]$, $A[i-2]$, $A[i-3]$ in registers, plus $2\times$ unrolling.

Kernel 3 ($h[idx[i]] += w[i]$): Indirect (scatter) access causes random cache misses on $h[]$. Two variants: (A) $4\times$ unrolling with register-cached indices, and (B) sorting $idx[]$ for sequential access to $h[]$.

Kernel 4 (`sum += diff if diff > 0`): Data-dependent branch ($\sim 50\%$ taken with random data). Optimized with branchless accumulation (`sum += (d > 0) ? d : 0`), 4 independent accumulators to hide FPU latency, and $4\times$ loop unrolling.

3 Results

Table 2 presents all measured execution times and speedups relative to the baseline. The “No FPU” row isolates the FPU contribution by disabling only the FPU while keeping all other hardware optimizations enabled.

Table 2: Execution times (seconds) and speedup ($\times$ vs. baseline) for each kernel.

Configuration	Kernel 1		Kernel 2		Kernel 3		Kernel 4	
	Time	S $\uparrow$	Time	S $\uparrow$	Time	S $\uparrow$	Time	S $\uparrow$
1. Baseline	0.0567	1.0	0.00916	1.0	0.5706	1.0	44.07	1.0
2. Compiler	0.0159	3.6	0.00738	1.2	0.5177	1.1	1.092	40
3. Hardware	0.00230	25	0.000092	100	0.00369	155	0.00304	14,479
4. Software	0.00268	21	0.000122	75	0.00341	167	0.00313	14,081
5. All combined	0.00138	41	0.000025	366	0.00186	307	0.00176	25,084
6. No FPU (HW opt)	N/A		N/A		0.01621	35	0.03091	1,426

The FPU alone accounts for a $10.2\times$ speedup on Kernel 4 (comparing rows 3 and 6), confirming that software floating-point emulation is the single largest bottleneck for FP-heavy workloads.

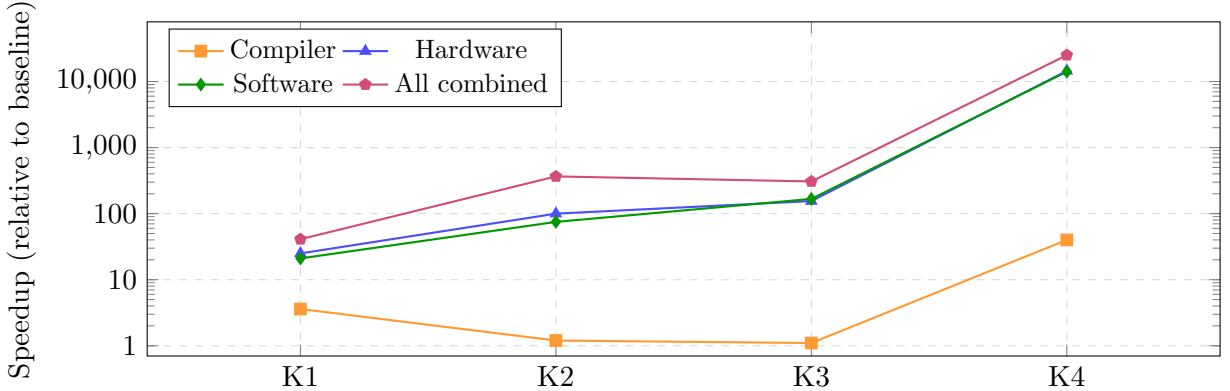


Figure 3: Speedup relative to baseline (log scale). Hardware optimization dominates; the compiler alone provides limited improvement. The combination of all three approaches yields the highest overall speedup.

4 Discussion

4.1 Hardware Dominates Performance

The most striking result is that hardware optimization alone (config 1 $\rightarrow$ 3) yields speedups of $25\text{--}14,479\times$, dwarfing compiler-only improvements ($1\text{--}40\times$). This demonstrates that the memory hierarchy is the primary performance bottleneck: without caches, every load/store incurs $\sim 30+$ cycle DDR3 latency, making even simple loops extremely slow. Kernel 4 shows the most extreme case ($14,479\times$) because it combines two effects: the FPU eliminates software floating-point emulation ($\sim 10\times$) and the cache eliminates ~ 4096 DDR3 accesses per array ($\sim 1400\times$).

To isolate the compute-unit contribution from caches, we also measured with full HW optimization but caches disabled. Kernel 2 achieved $29\times$ speedup ($0.007166\text{ s} \rightarrow 0.000248\text{ s}$) purely from the hardware multiplier, and Kernel 4 achieved $71\times$ ($1.092\text{ s} \rightarrow 0.0154\text{ s}$) from the FPU alone. This

confirms that compute units and caches address fundamentally different bottlenecks: arithmetic throughput vs. memory latency.

4.2 Compiler Optimization Without Cache Is Limited

Configuration 2 shows that `-O2` provides only moderate improvement without caches (K2: $1.2\times$, K3: $1.1\times$). The compiler can schedule instructions optimally, but when every load stalls for 30+ cycles, instruction-level improvements are overwhelmed by memory latency. The notable exception is K4 ($40\times$), where the compiler replaces software FP emulation library calls with inlined sequences. This confirms that **compiler optimization cannot overcome the memory wall without hardware support**.

4.3 Software Optimization: Context-Dependent

An unexpected finding: at `-O0` with full hardware optimization, the software-optimized code is marginally *slower* for K1 and K2 (configs 3 vs. 4). This occurs because at `-O0`, the additional local variables (pointer temporaries, register window values) are placed on the stack rather than in registers, adding overhead. However, when combined with `-O3` (config 5), the compiler can fully utilize the restructured code, yielding the best overall performance.

A further subtlety: at `-O3` with full HW, Kernel 1’s optimized code is actually *slower* than the baseline (0.00138s vs. 0.00114s, speedup $0.83\times$). This demonstrates that `-O3` already applies the same loop unrolling and pointer optimizations automatically. The hand-optimized code then adds redundant structure that the compiler cannot fully simplify, resulting in a net negative. This reinforces that manual software optimization is most valuable at low optimization levels or when caches are absent.

Crucially, when caches are *disabled* but other HW features are present, the software optimizations provide significant gains (K1: $1.79\times$, K2: $1.70\times$ at `-O0`), because each eliminated load saves 30+ cycles of DDR3 latency. This confirms the optimizations are correctly targeted at memory access reduction.

4.4 Why Some Kernels Are Easier to Optimize

Kernel 1 (easiest): Regular stride-1 access pattern benefits enormously from caching and unrolling. No dependencies between iterations allow full exploitation of all optimization techniques.

Kernel 2 (hardest for software): The loop-carried dependency ($A[i]$ depends on $A[i-1]$) fundamentally limits parallelism. However, the register sliding window is highly effective because it targets loads, not computation.

Kernel 3: The indirect access pattern makes this kernel cache-hostile. The sort-based optimization (variant B) was counterproductive: $O(n^2)$ insertion sort on 4096 elements costs far more than it saves, since `h[]` (4 KB) fits entirely in the 8 KB data cache. The unrolled variant provides modest improvement by reducing branch overhead on the sequential `idx[]/w[]` accesses.

Kernel 4: Benefits most dramatically from hardware (FPU + cache). The branchless optimization shines at `-O3` with BTC enabled ($1.46\times$), showing that software branch elimination and hardware branch prediction address the same bottleneck from different angles.

4.5 Key Takeaway

The optimal strategy is a combination of all three layers: hardware provides the foundation (cache, FPU, BTC), software restructuring reduces unnecessary work (register reuse, branch elimination), and the compiler translates well-structured code into efficient machine instructions. No single layer is sufficient alone.

A Additional Figures

Figure 4 shows execution times for all four kernels on a shared logarithmic y-axis. A shared axis enables direct cross-kernel comparison: after full optimization, all kernels converge to similar magnitudes (~ 1 ms), despite baselines spanning three orders of magnitude.

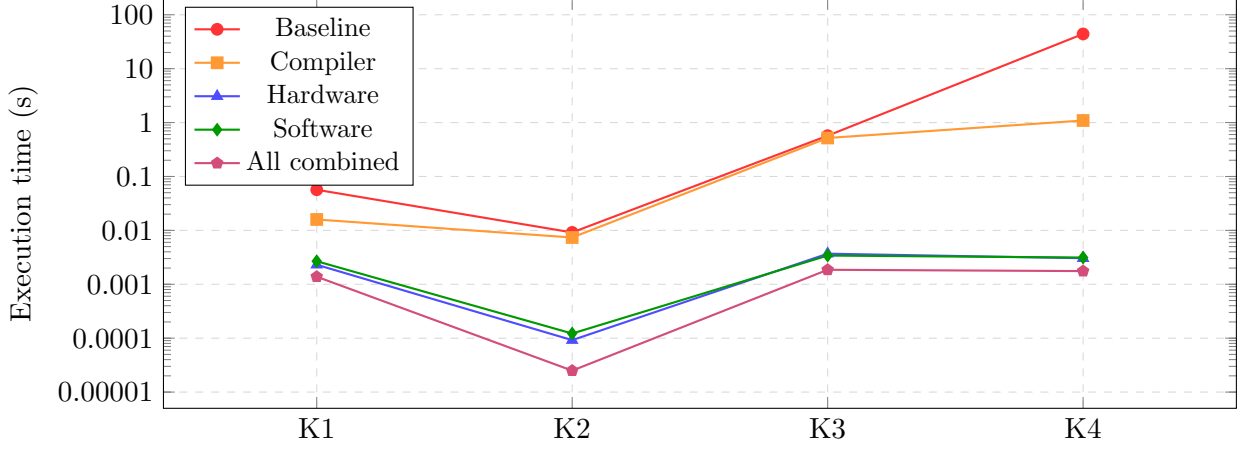


Figure 4: Execution times across all kernels (shared log scale). Higher points indicate slower execution. All kernels converge to ~ 1 – 2 ms after full optimization, despite baselines spanning three orders of magnitude.

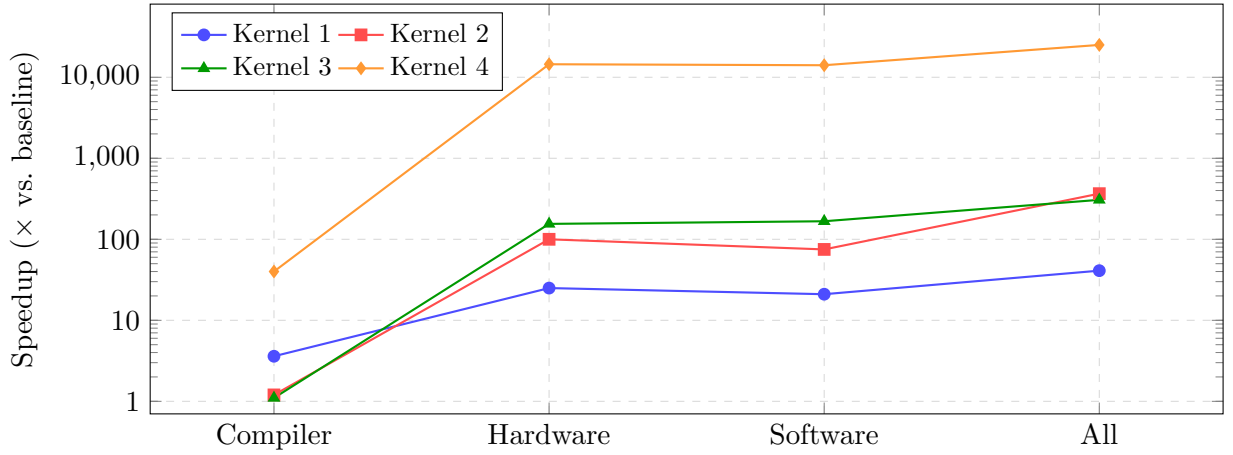


Figure 5: Per-kernel speedup by optimization type (shared log scale). Kernel 4 gains the most from hardware (FPU), while Kernel 2 shows the strongest all-combined multiplier due to HW multiplier, caches, and register optimization together.